



Python Basics

By: Devesh Sir

What is a Python?

Python is a Programming Language

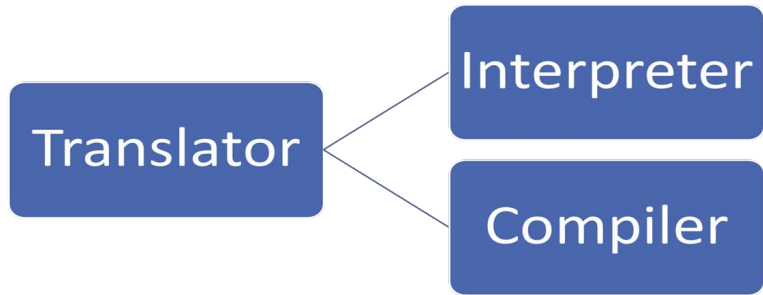
What is a Programming language ?

Programming Language is a medium of communication between human and computer.

“A programming language is collection of symbols and formal strings with pre-defined meaning and which has its own way of writing which is called syntax.”

Translator

A translator in programming is a program which converts our code called source code to machine code.



Interpreter vs Compiler

Interpreter

“An interpreter is a program that translates high-level code into machine code line-by-line, executing each line as it translates.”

Compiler

“A compiler is a program that translates the entire high-level code into machine code at once, creating an executable file that can be run separately.”

Generation of Programming language

1. **First Generation** – Machine Language -Written in binary (0s and 1s), directly understood by the computer but very hard for humans to read or debug.
2. **Second Generation** – Assembly Language -Uses symbolic codes (mnemonics) instead of binary, easier to read but still hardware-dependent and needs an assembler.
3. **Third Generation** – High-Level Language - Uses English-like syntax, machine-independent, and requires a compiler or interpreter. Examples: C, Java, Python.
4. **Fourth Generation** – Very High-Level Language - Closer to human language, needs less coding, mainly used for databases and report generation. Example: SQL.
5. **Fifth Generation** – Artificial Intelligence Language - Used for AI and machine learning; focuses on what to do, not how. Examples: Prolog, LISP, Python (AI).

Why Python for Data Analysis

- **Easy to learn** – You can focus on Data Analysis rather than programming.
- **Powerful Libraries** - It has powerful libraries like Pandas, Matplotlib, Numpy, Seaborn.
- **Integration with Other Tools:** Python easily integrates with other data sources, tools, and technologies like SQL databases, Excel, and machine learning frameworks.

Basic Terms

Basic Terms in Python

- **Keywords:** keywords are reserved words that have special meaning to the Python language. They are used to define the syntax and structure of Python programs, and you cannot use them as variable names, function names, or identifiers.
- **Indentation :** indentation means adding spaces or tabs at the beginning of a line to define blocks of code. Unlike many other programming languages (like C or Java) that use { } to mark code blocks, Python uses indentation to show which statements belong together.
- **Syntax :** syntax is the set of rules that define how programs must be written so the computer can understand and run them correctly.

variable

a **variable** is a name that you use to store data or a value in memory so that you can use it later in your program.

It acts like a container that holds some information — for example, a number, text, or list — and allows you to refer to that data by name instead of writing it directly every time.

```
x = 10  
name = "Alice"
```

Rules for naming variables

- Variable names can include letters(a-z,A-Z), numbers(0-9), and underscores (_).
- They must start with a letter or underscore, not a number.
- Variable names are case-sensitive (Name and name are different).
- Don't use Python keywords (like if, for, class, etc.) as variable names.

Delete a Variable

```
a = 100  
del a  
print(a)
```

NameError: name 'a' is not defined

Arithmetic Operators

```
a = 10
```

```
b = 3
```

```
print(a + b)  # Addition → 13
```

```
print(a - b)  # Subtraction → 7
```

```
print(a * b)  # Multiplication → 30
```

```
print(a / b)  # Division → 3.333...
```

```
print(a // b) # Floor Division → 3
```

```
print(a % b)  # Modulus (remainder) → 1
```

```
print(a ** b) # Exponentiation → 1000
```

Comparison (Relational) Operators

```
x = 5
```

```
y = 10
```

```
print(x == y)  # Equal to → False
```

```
print(x != y)  # Not equal to → True
```

```
print(x > y)    # Greater than → False
```

```
print(x < y)    # Less than → True
```

```
print(x >= y)   # Greater than or equal to → False
```

```
print(x <= y)   # Less than or equal to → True
```

Logical Operators

```
a = True
b = False
print(a and b)  # True if both are True → False
print(a or b)   # True if at least one is True → True
print(not a)    # Reverses True to False → False
```

Assignment Operators

```
x = 10
```

```
x += 5 # x = x + 5 → 15
```

```
x -= 3 # x = x - 3 → 12
```

```
x *= 2 # x = x * 2 → 24
```

```
x /= 4 # x = x / 4 → 6.0
```


Membership Operators

Used to test whether a value is present in a sequence (like list, string, etc.).

```
fruits = ["apple", "mango"]  
print("apple" in fruits)      # True  
print("banana" not in fruits) # True
```

Print Function

the `print()` function is used to display output on the screen. It allows you to show text, numbers, or the result of an expression when your program runs.

```
print("Hello, World!")
```

```
name = "Alice"  
age = 25  
print("Name:", name, "Age:", age)
```

```
print(10 + 5)  
print("Sum:", 10 + 5)
```

Print - Sep and End Arguments

sep decides what separates multiple items in the output (default is a space).

```
print("Python", "is", "fun", sep="-")
```

Python-is-fun

end controls what happens at the end of each print statement (default is a new line \n).

```
print("Hello", end=" ")  
print("World")
```

Hello World

Printing with formatted strings (f-strings)

This lets you insert variables directly inside strings.

```
name = "Bob"  
age = 30  
print(f"My name is {name} and I am {age} years old.")
```

```
name = "shivam"  
age = 56  
text = f"my name is {name} and age is {age}"  
print(text)
```

Input Function

the `input()` function is used to take input from the user during program execution. It allows your program to pause and wait until the user types something on the keyboard, and then it stores that input as a string.

```
variable = input("Message for user: ")
```

Questions

1. Write a Python program to add two numbers entered by the user.
2. Write a program to calculate the square and cube of a number.
3. Write a program to calculate the area of a rectangle (length $\times$ width).
4. Write a program to calculate the area of a circle using radius = 7.
5. Write a program to find the average of three numbers.
6. Write a program to convert temperature from Celsius to Fahrenheit.
7. Write a program to calculate the simple interest (use formula: $SI = (P * R * T) / 100$).
8. Write a program to calculate the compound interest.
9. Write a program to find the square root of a number using exponent operator (**).
10. Write a program to find the perimeter of a square and a rectangle.

Questions

1. Write a program to check if a number is even or odd.
2. Write a program to swap two numbers using a temporary variable.

Conditional Statements

IF Function

if statement is used to make decisions in a program. It lets your program check a condition and run certain code only if that condition is true. If the condition is false, the code inside the if block will be skipped.

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

Without Else



```
age = 16
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

With Else



Questions for if

1. Write a program to check whether a number is positive or negative.
2. Write a program to check if a number is even or odd.
3. Write a program to print “Welcome” if the user enters the correct password.
4. Write a program to check whether a person is eligible to vote (age ≥ 18).
5. Write a program to check if a given number is divisible by 5.
6. Write a program to check if a number is greater than 100 or not.

Questions for If Else

1. Write a program to check if a number is even or odd.
2. Write a program to check whether a year is a leap year or not.
3. Write a program to find the largest of two numbers.
4. Write a program to check whether a character is a vowel or consonant.
5. Write a program to check whether a person can drive (age ≥ 18).
6. Write a program to check whether a number is divisible by both 3 and 5.

If - Elif - Else

Used when you have multiple conditions to check.

```
marks = 85

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: D")
```

Questions for If - Elif - Else

1. Write a program to print grades based on marks:
 - a. 90–100 → A
 - b. 75–89 → B
 - c. 60–74 → C
 - d. Below 60 → Fail
2. Write a program to check whether a given number is positive, negative, or zero.
3. Write a program to display the day of the week (1–7 → Monday–Sunday).
4. Write a program to find the largest among three numbers.
5. Write a program to check the type of triangle (equilateral, isosceles, scalene) based on sides.
6. Write a program to determine the category of a person based on age (child, teenager, adult, senior).

Nested if

```
num = int(input("Enter a number: "))

if num > 0:
    print("The number is positive.")
    if num % 2 == 0:
        print("It is even.")
    else:
        print("It is odd.")
elif num < 0:
    print("The number is negative.")
    if num % 2 == 0:
        print("It is even.")
    else:
        print("It is odd.")
else:
    print("The number is zero, and it is neither even nor odd.")
```

Questions for Nested If

1. Write a program to check whether a number is positive, and if positive, check whether it is even or odd.
2. Write a program to check if a student has passed (marks ≥ 40), and if passed, check if they scored distinction (marks ≥ 75).
3. Write a program to check if a person is eligible for a loan — they must be above 21 and have a salary greater than ₹20,000.
4. Take length and width from user and print if its a square or rectangle if its a rectangle then if its a height is more or its width is more?

Loops

For loop

A for loop in Python is used to repeat a block of code a specific number of times

When use range



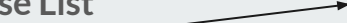
```
for i in range(5):  
    print(i)
```

When use Text



```
for letter in "Python":  
    print(letter)
```

When use List



```
numbers = [10, 20, 30, 40, 50]  
  
for num in numbers:  
    print("The number is:", num)
```

Questions for for loop

1. Write a program to print numbers from 1 to 10 using a for loop.
2. Write a program to print each character of the string "Hello world" using a for loop.
3. Write a program to print the sum of numbers from 1 to 50.
4. write a function to check if the number is prime or not
5. print all the even numbers from 1 to 50.
6. Count how many 'O' are in Hello World
7. find the factorial of a given number
8. Print a square shape using stars "*"
9. print a triangle shape using stars "*"
10. Print a inverted triangle shape using stars ?
11. Write a function to find how many vowels are in that text(*give any random text*)?

While loop

A while loop in Python is used to repeat a block of code as long as a certain condition is true.

```
while count <= 5:  
    print("Count is:", count)  
    count += 1
```



```
Count is: 1  
Count is: 2  
Count is: 3  
Count is: 4  
Count is: 5
```

```
age = int(input("Enter your age"))  
while age > 100:  
    age = int(input("Please enter your correct age"))  
print(f"Your age is {age}")
```

Questions for While loop

1. Create a login window where a person will enter a password and if he gives a correct password then print welcome and retake the password unlimited number of times until he gives you the real password?

Data Type: Strings

Data Types in Python

1. `int` (Integer) – Used for whole numbers without decimals.
2. `float` (Floating Point) – Used for numbers with decimals
3. `complex` – Used for complex numbers with a real and imaginary part.
4. `str` (String) – Used for text or sequence of characters.
5. `bool` (Boolean) – Used for logical values `True` or `False`.

Data Structures

6. `list` – Used to store multiple items in one variable, ordered and changeable.
7. `tuple` – Similar to a list but cannot be changed (immutable).
8. `set` – Used to store unordered and unique items.
9. `dict` (Dictionary) – Used to store data in key-value pairs.

Type Function

you can check the data type of any variable using the `type()` function. The `type()` function returns the class or type of the value stored in the variable.

- `type(12)` `# int → integer number`
- `type(3.14)` `# float → decimal number`
- `type(2 + 3j)` `# complex → complex number`
- `type(True)` `# bool → boolean value`
- `type("Hello")` `# str → string (text)`
- `type([1, 2, 3])` `# list → ordered, changeable collection`
- `type((1, 2, 3))` `# tuple → ordered, unchangeable collection`
- `type({1, 2, 3})` `# set → unordered collection of unique items`
- `type({"name": "Devesh"})` `# dict → key-value pair collection`
- `type(None)` `# NoneType → represents "nothing" or "no value"`

Type Casting

type casting (also called type conversion) means changing the data type of a value or variable from one type to another. Python provides built-in functions to convert between types like int, float, str, and others.

- `int()` → converts to integer
- `float()` → converts to float
- `str()` → converts to string
- `bool()` → converts to boolean (True or False)
- `list()` → converts to list
- `tuple()` → converts to tuple
- `set()` → converts to set

Int , Float, Complex and Bool

- **INT** : (short for integer) data type is used to store whole numbers — that is, numbers without any decimal point.
 - Ex : 23, 67, 9, 0, -12, -45
- **Float** : (Floating Point Numbers) float type represents numbers with decimal points
 - Ex: 2.987, 3.556, 4.5
- **Complex** : The complex type is used to represent complex numbers, which have a real part and an imaginary part. A complex number is written as $a + bj$
 - Ex: $3+6j$, $45+2j$
- **Bool**: (Boolean) data type represents truth values — either **True** or **False**. It is often used in conditional statements and logical operations.

Strings

- A string in Python is a sequence of characters enclosed in quotes.
- You can use single (' '), double (" "), or triple quotes (''' ' or '""" ""') for multi-line strings.
- **Strings are immutable**, meaning once created, their content cannot be changed.
- Each character in a string has order an index, starting from 0.
- You can access, slice, and loop through strings easily.

```
name = "Devesh"  
sentence = 'Python is fun'  
paragraph = """This is  
a multi-line  
string."""
```

Indexing and Slicing

indexing and slicing are ways to access parts of sequences such as strings, lists, or tuples.

	Forward direction indexing					
	0	1	2	3	4	5
String	p	y	t	h	o	n
	-6	-5	-4	-3	-2	-1
	Backward direction indexing					

```
a = "hello world"
a[1:5] # 'ello'
a[0:2] # 'he'
a[8:11] # 'rld'
```

String Methods

- **upper()** – Converts all characters to uppercase.
- **lower()** – Converts all characters to lowercase.
- **title()** – Capitalizes the first letter of each word.
- **capitalize()** – Capitalizes only the first letter of the string.
- **strip()** – Removes spaces from the beginning and end.
- **replace(old, new)** – Replaces part of a string with another string.
- **split(separator)** – Splits the string into a list based on a separator.
- **join(iterable)** – Joins elements of a list or tuple into one string. Ex : “ - “.join(list)

String Methods

- **find(substring)** – Returns the index of the first occurrence of a substring (or -1 if not found).
- **count(substring)** – Counts how many times a substring appears.
- **isalpha()** – Returns True if all characters are alphabets.
- **isdigit()** – Returns True if all characters are digits.
- **isalnum()** – Returns True if all characters are alphanumeric (letters or numbers).

Practice Questions

1. You have to add $a = \text{"3hj4h5g3fep"}$, $b = \text{"kg75yfde8"}$? -ans **(3453+758) = 4211**
2. Separate digit, text and special from $a = \text{"3248wdhfh8347h7sfd*T\^g\^R\^76@\#\%\$"}$
3. Write a program to check whether a character is a letter, and if it is, check whether it's uppercase or lowercase
4. Count how many vowels are in this text $a = \text{"this is a random text"}$

List

a list is a collection data type that **can store multiple items in a single variable**. It is one of the most commonly used and flexible data structures in Python. A list is written using square brackets [], and the items inside it are separated by commas.

- **Ordered** – The items have a defined order and can be accessed using an index.
- **Mutable** – You can change, add, or remove items after creating the list.
- **Allows duplicates** – Lists can contain duplicate values.
- **Can store mixed data types** – A list can have strings, integers, floats, and even other lists.

```
fruits = ["apple", "banana", "cherry"]  
print(fruits)
```

Mutable And Immutable

A mutable object is one whose data can be modified in place without creating a new object. This means the object's identity (memory address) stays the same, but its contents can change. Common mutable objects in Python include lists, dictionaries, sets, and bytearray. For example, if you have a list and you change one of its elements, the same list object is updated rather than creating a new list.

An immutable object is one whose value cannot be changed after it is created. If you try to modify an immutable object, Python creates a new object instead of changing the existing one. Common immutable objects include integers, floats, strings, tuples, and frozensets. For example, when you change a string, Python actually creates a new string rather than modifying the original one.

List

- **Indexing** : You can access list elements using index numbers

```
numbers = [10, 20, 30, 40]
print(numbers[0])    # Output: 10
print(numbers[-1])   # Output: 40
```

- **Slicing** : You can extract a portion of the list using slicing

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])   # Output: [20, 30, 40]
print(numbers[:3])    # Output: [10, 20, 30]
print(numbers[::2])    # Output: [10, 30, 50]
```

Methods of List

- **append(x)** – Adds an item x at the end of the list.
- **extend(iterable)** – Adds multiple elements (like another list or tuple) at the end.
- **insert(i, x)** – Inserts an item x at the specified index i.
- **remove(x)** – Removes the first occurrence of the value x.
- **pop(i)** – Removes and returns the element at index i. if no index is given it removes the last item.
- **clear()** – Removes all elements from the list, making it empty.
- **index(x)** – Returns the index of the first occurrence of value x.
- **count(x)** – Returns how many times value x appears in the list.
- **sort()** – Sorts the list in ascending order by default.
- **reverse()** – Reverses the order of elements in the list.
- **copy()** – Creates and returns a copy (duplicate) of the list.

Questions for List

1. You have to create a list of names in which the number of participants are unknown when the person tell you to end the list then you will stop asking the names At the last you have to print the name list
2. Create a list of subjects names then create another list of marks and you have to insert marks on the basis of subject name
3. amount = ["\$45","\$77","\$12","\$90"] - input list
output - [45,77,12,90] - create a new list with these values
4. a = [3,4,5,4,3,4,3,4,3,3,3,5] - from this list remove all the 3?
5. a = [3,4,5,6,5,4,5,6,5,4,1,8] - remove all the duplicate element from this list using for loop

Tuples

A tuple in Python is an ordered, immutable collection of items. This means the elements inside a tuple are stored in a specific order and cannot be changed, added, or removed after the tuple is created. Tuples are written using parentheses (), and their elements can be of different data types (integers, strings, lists, etc.).

Ex: `my_tuple = (10, 20, 30)`

Methods in Tuple :

- `count(x)` – Returns how many times the value x appears in the tuple.
- `index(x)` – Returns the index (position) of the first occurrence of the value x.

Set

A set is a collection data type used to store multiple unique items in a single variable. Sets are **unordered**, **unindexed**, and **do not allow duplicate values**. You create a set using curly braces `{ }` or the `set()` function.

- **Unordered** – The items don't have a fixed position or index.
- **Unindexed** – You can't access elements using indexes like lists.
- **No duplicates** – Duplicate items are automatically removed.
- **Mutable** – You can add or remove elements, but cannot change existing ones.
- **Heterogeneous** – Can store different data types (numbers, strings, etc.).
- A set will store only immutable objects

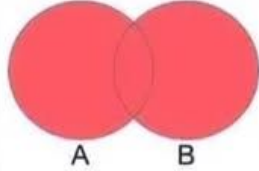
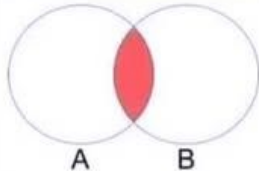
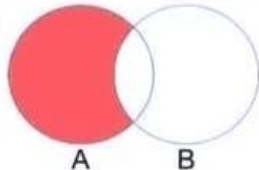
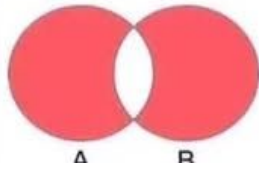
Ex: `fruits = {"apple", "banana", "cherry"}`

Set Methods

- **add()** – Adds an element to the set.
- **update()** – Adds elements from another set to the current set.
- **remove()** – Removes a specified element from the set (raises error if not found).
- **discard()** – Removes a specified element from the set (no error if not found).
- **pop()** – Removes and returns a random element from the set.
- **clear()** – Removes all elements from the set.

Set Operators

- **| (Union)** – Returns all unique elements from both sets.
- **& (Intersection)** – Returns common elements from both sets.
- **- (Difference)** – Returns elements present in the first set but not in the second.
- **^ (Symmetric Difference)** – Returns elements present in either set but not in both.

Union		$A \cup B$, is the set of all values that are a member of A , or B , or both.
Intersection		$A \cap B$, is the set of all values that are members of both A and B .
Difference		$A \setminus B$, is the set of all values of A that are not members of B
Symmetric Difference		$A \triangle B$, is the set of all values which are in one of the sets, but not both.

Dictionary

dictionary is a data type used to store data in the form of key-value pairs. It is written inside curly brackets {}, where each key is separated from its value by a colon : Dictionaries are

- Unordered
- Mutable (you can change them)
- Don't allow duplicate keys.

Ex: `student = {"name": "Ravi", "age": 25, "city": "Bhopal"}`

Operations in Dictionary

- `student["name"]` -> Get values from a Dict
- `student["course"] = "Data Science"` -> Add new key-value pair
- `student["age"] = 26` -> Update value

Dictionary Methods:

- `dict.keys()` → returns a view object with all keys
- `dict.values()` → returns a view object with all values
- `dict.items()` → returns a view object with all key-value pairs
- `dict.get(key, default)` → returns the value for the given key; returns default if key not found
- `dict.pop(key, default)` → removes the item with the given key and returns its value; returns default if key not found

Operations in Dictionary

1. Create a dictionary with keys from 1 to 10 in their value as their square?
2. Create a dictionary school subject with name like Math, Science,economics etc as keys and with their marks as their value?
3. Using the above dictionary find the total marks, average marks, subject with the minimum Marks and subject with the maximum marks?
4. You are given a list ['name', 'age', 'gender', 'salary']. Using a for loop, create a dictionary where each key is taken from this list, and the corresponding value is entered by the user. Finally, print the dictionary.
5. Using this list [3,4,5,6,7,8,9,12,14,15] - make a dict like this {3:"odd",4:"even",5,"odd..."}

Functions

a function is a block of code that performs a specific task and can be reused whenever needed. It helps make programs shorter, cleaner, and easier to manage. A function is defined using the `def` keyword followed by the function name, parentheses (which may include parameters), and a colon. The code inside the function is indented.

Types of functions

1. **Built-in Functions**
2. **User-defined Functions**
3. **Lambda (Anonymous) Functions**
4. **Recursive Functions**

Built-in Functions

Built-in functions in Python are the functions that come **predefined with Python**, meaning you can use them without importing any library or writing any extra code. They are always available and help perform common tasks such as mathematical operations, data type conversions, input/output, and more.

• print • input • type • id • range	• int • float • str • bool • list • tuple • dict • set	• sorted • map
---	--	--

User Defined Functions

User-defined functions in Python are the functions that you create yourself to perform specific tasks. They help make your code more organized, reusable, and easier to understand. You define a function using the `def` keyword, give it a name, optionally add parameters, and then write the code that runs when the function is called.

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)  # Output: 8
```

Doc String

A docstring (documentation string) in Python is a string literal used to explain the purpose and behavior of modules, classes, functions, or methods. Defined using triple quotes (""") immediately following the object's definition, it becomes part of the code's metadata, making it accessible at runtime via `__doc__` or `help()`.

global and a local variable

a local variable is a variable that is created inside a function and can be used only within that function. It exists only while the function is running, and once the function finishes execution, the local variable is destroyed. This means you cannot access a local variable outside the function in which it is defined.

A global variable, on the other hand, is a variable that is defined outside of all functions, usually at the top level of a program. A global variable can be accessed from anywhere in the code, including inside functions.

```
a = 100
def hello():
    b = 500 # local variable
    global c # global variable
    c = 999
    print(b+a)
```

global and a local variable

if you want to modify a global variable inside a function, you must explicitly declare it using the global keyword; otherwise, Python will treat it as a local variable within that function.

Functions Questions

1. WAF to calculate simple interest take amount , rate and year as arg?
2. WAF to calculate compound interest using amount, rate and year?
3. WAF to find the factorial of a given number?
4. WAF to find if a given number is prime or not?
5. WAF to print the table of given number like (4 X 2 = 8) this format?
6. WAF to find if a number is even or odd?
7. WAF to greet according to gender

`greet(name.gender)` → Hello Mr. Rohan

8. WAF to check is a word is a palindrome or not?
9. What is the difference when you use return and print in a function?

Types of parameters

parameters are the variables you define inside a function to accept input values when the function is called. There are five main types of parameters in Python functions:

1. **Positional Parameters** : These are the most common type. The values are passed to the function in the same order as the parameters are defined.

```
def add(a, b):  
    print(a + b)  
  
add(5, 3)  # a = 5, b = 3
```

Types of parameters

2. Default Parameters : You can assign a default value to a parameter. If the caller doesn't provide a value, the default is used.

```
def greet(name="User"):  
    print("Hello,", name)
```

3. Keyword Parameters: When calling a function, you can specify the parameter name explicitly

```
def info(name, age):  
    print(name, "is", age, "years old")  
  
info(age=25, name="Ravi")
```

Types of parameters

4. Arbitrary parameters: Use an asterisk * before a parameter name to accept any number of positional arguments. The function receives them as a tuple.

*Ex: def total(*nums):*

5. Arbitrary Keyword Parameters (kwargs):** Use double asterisks ** to accept any number of keyword arguments. The function receives them as a dictionary.

*Ex : def details(**info):
 details(name="Ravi", age=25, city="Delhi")*

Lambda Function

A lambda function in Python is a small, anonymous function — meaning it doesn't have a name like regular functions defined using `def`. It is mainly used when you need a simple function for a short period of time. You create it using the `lambda` keyword.

- It can take any number of arguments.
- It can only contain one expression (no multiple lines).
- It automatically returns the result of that expression.

lambda arguments: expression

```
add = lambda a, b: a + b  
print(add(3, 7))    # Output: 10
```

Recursion Function

Recursion function

```
def whats_your_age():  
    num = int(input("what is your age"))  
    if num<=100:  
        print("your age is ",num)  
    else:  
        whats_your_age()
```

```
whats_your_age()
```

```
what is your age 999
```

```
what is your age 6778
```

```
what is your age 9
```

```
your age is 9
```

Ternary Operator

shortcut method of writing if in Python

Syntax:

```
value_if_true if condition else value_if_false
```

Example:

```
result = "Pass" if marks >= 40 else "Fail"
```

List comprehension

List comprehension in Python is a short and clean way to create a list using a for loop.

```
squares = [x * x for x in range(1, 6)]
```

```
print(squares)
```

```
# Output: [1, 4, 9, 16, 25]
```


Modules And Library

Module : A module is simply a single Python file (.py) that contains functions, variables, or classes which you can import and use in another program. **Ex: random, time, math**

Library : A library is a collection of multiple modules bundled together to provide a wide range of functionalities or **library = many modules combined** to solve specific kinds of problems. **Ex: Numpy, Pandas, Seaborn etc..**

To import them use:

1. `import random` - this will import everything as random
2. `import random as r` - this will import everything as r
3. `from random import randint` - only import randint functions from random

Useful functions

1. Time module :

- a. `time.time()` - Returns the current time in seconds since January 1, 1970
- b. `time.sleep(seconds)` - Pauses program execution for the given number of seconds.
- c. `time.ctime()` - Converts the current time into a readable string format.

2. Random Module

- a. `random.random()` - Returns a random float between 0.0 and 1.0.
- b. `random.randint(a, b)` - Returns a random integer between a and b (inclusive).
- c. `random.choice(sequence)` - Randomly changes the order of elements in a list.

Welcome to the "Guess My Number" Game!

In this Python program, you'll write a simple and fun number guessing game against the computer.

How the Game Works:

- The computer will randomly select a number between 1 and 100.
- Your task is to guess the number chosen by the computer.
- After each guess:
 - If your guess is correct, the computer will congratulate you.
 - If your guess is incorrect, the computer will tell you whether to guess a higher or lower number.
- The computer will keep track of the number of attempts you make.
- The game will continue until you guess the correct number.
- Once you find the correct number, the computer will display a congratulatory message along with the
- total number of attempts you took.